

Concordia: Enabling Low-Conflict Distributed Transaction Scheduling in Sharding Blockchain via Cooperative Perception

Yanxiu Liu

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China
University of Chinese Academy of
Sciences
Beijing, China
liuyanxiu20z@ict.ac.cn

Linpeng Jia

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China
State Key Laboratory of Blockchain
and Data Security, Zhejiang
University
Hangzhou, China
jialinpeng@ict.ac.cn

Xiaohu Yang

State Key Laboratory of Blockchain
and Data Security, Zhejiang
University
Hangzhou, China
yangxh@zju.edu.cn

Zhongcheng Li

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China
University of Chinese Academy of
Sciences
Beijing, China
zcli@ict.ac.cn

Yi Sun*

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China
University of Chinese Academy of
Sciences
Beijing, China
sunyi@ict.ac.cn

Abstract

Sharding is a key approach to scale blockchain, where transaction scheduling critically impacts performance. Traditional sharding uses non-perception scheduling, where each shard makes decisions independently, leading to high conflicts. Although some studies use a centralized scheduler to eliminate conflicts by gathering full shard data, but incur security risks. To address this, we propose Concordia, a distributed transaction scheduling scheme via cooperative perception. Each shard predicts future cross-shard interactions via a Graph Neural Network (GNN) based algorithm, generates low-dimensional perception signals, and shares them through incremental personalized broadcasts. Shards then use a popularity-driven transaction packing algorithm to schedule and pack transactions effectively. With 16 shards, Concordia achieves 2.39× higher throughput than non-perception scheduling while cutting conflicts from 62.3% to 1.5%, and improves throughput 1.67× over centralized scheduling while reducing perception overhead by 95.2%.

CCS Concepts

• Computer systems organization → Peer-to-peer architectures;

Keywords

Blockchain, Sharding, Transaction scheduling

*Corresponding authors



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792469>

ACM Reference Format:

Yanxiu Liu, Linpeng Jia, Xiaohu Yang, Zhongcheng Li, and Yi Sun. 2026. Concordia: Enabling Low-Conflict Distributed Transaction Scheduling in Sharding Blockchain via Cooperative Perception. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792469>

1 Introduction

Blockchain, as a foundational infrastructure of the Web3.0 era [2, 14, 18], directly supports decentralized web services. However, its poor performance limit user experience of decentralized Web applications. Sharding [17, 27] offers a promising solution to enhance blockchain performance by partitioning the network into multiple shards to process transactions (TXs) in parallel. Yet, sharding introduces cross-shard transactions (CTXs) when accounts span multiple shards, requiring coordinated processing between shards. However, current studies [10, 13, 34, 40] mainly focuses on ensuring CTX atomicity across shards, with little attention to TX scheduling, which determines how shards select and pack TXs into blocks. In fact, transaction scheduling is crucial to system performance, as unreasonable scheduling can cause conflicts when multiple CTXs access the same account, severely degrading performance.

Most existing sharding [17, 23, 33] adopt a **non-perception scheduling** scheme, where each shard independently packs TXs based on its limited local view, as shown in Fig. 1(a). For example, in Fig. 2, CTX 2 conflicts with CTX 1 by modifying account B before CTX 1 commits. Lacking information from other shards, such a scheme fails to achieve globally optimized TX scheduling, resulting in frequent conflicts with conflict ratios exceeding 60% in our measurements (in Section 7). Recent studies [1, 11] propose a **centralized perception scheduling** scheme as shown in Fig. 1(b), where a central scheduler (a specific shard or node) collects

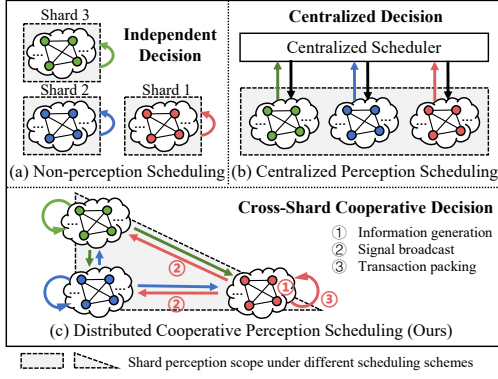


Figure 1: Illustration of different TX scheduling schemes.

all shards' account states and transaction pools (Txpools) to make global conflict-free scheduling decisions [11] for each shard. However, this design suffers from centralized security risks [28, 36] and, as shard numbers increase, frequent global synchronization creates severe performance bottlenecks.

To overcome the security risks and performance bottleneck of the centralized scheme, an ideal solution is a distributed scheduling approach, where shards share local information to form cross-shard perception and make coordinated scheduling decisions. However, no effective solution has been proposed due to three challenges:

Challenge-1. Information Representation: *how to extract compact yet informative perception information within each shard.* In a distributed scheduling scheme, shards must share local insights to broaden perception for better decisions. Since each shard holds vast data (e.g., transactions, account states, and runtime states), it is crucial to identify the most valuable information and generate lightweight representations for efficient guidance.

Challenge-2. Information Transmission: *how to transmit shard-specific information while keeping it fresh with low-overhead.* Accurate scheduling needs timely information sharing, but frequent updates are costly. When a shard shares information, the broadcast mechanism must balance information freshness and overhead, while adapting to varying cross-shard trends of each shard.

Challenge-3. Information Utilization: *how to leverage information for globally optimized TX scheduling.* Perception information from different shards reflects their local views of system states and TX trends. However, for a single shard, integrating this heterogeneous and dynamic information to form an accurate global view and optimize TX packing for global scheduling is challenging.

To solve above challenges, we propose Concordia, a distributed **cooperative perception scheduling** scheme in Fig. 1(c). In Concordia, each shard predicts its future cross-shard interactions and encodes them as low-dimensional scheduling signals broadcast to others. Using these signals, shards make distributed, system-aware scheduling decisions, enabling proactive global optimization to reduce conflicts, balance load, and improving performance. Our contributions are summarized as follows:

- **Graph Neural Network-based Information Generation Mechanism.** To predict a shard's future cross-shard trends, we encode the cross-shard spatio-temporal behaviors of hotspot accounts into low-dimensional information with a Graph Neural Network (GNN)-based algorithm.

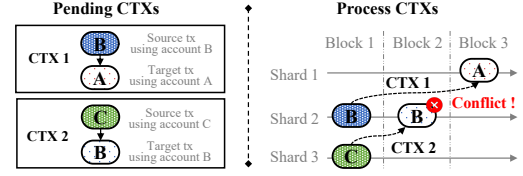


Figure 2: TX conflicts in non-perception scheduling sharding.

- **Incremental Personalized Broadcast Mechanism.** To enable low-overhead cross-shard sharing, we propose an incremental personalized broadcast mechanism that transmits only relevant account signals to each shard.
- **Popularity-driven TX Packing Algorithm.** We leveraging cross-shard perception for intra-shard TX packing algorithm, which formulates scheduling as a gain maximization problem by two-phase optimization with initial ranking and load-balanced sequence refinement.
- **Cooperative Perception Sharding Prototype.** We implement a Concordia sharding prototype and evaluate it using real Ethereum TXs [39]. With 16 shards, Concordia achieves 2.39× higher throughput than non-perception scheduling and cuts packed transaction conflicts from 62.3% to 1.50%. Compared to centralized scheduling, it boosts throughput by 1.67× while lowering perception overhead by 95.2%.

2 Measurements and Observations

2.1 Analysis of Transaction Network Topology

First, we analyze the topological characteristics of the transaction network. We use an Ethereum [39] TX dataset consisting of 3 million TXs. We construct the account-transaction graph where accounts are nodes and TXs are edges, and define a node's degree as the number of connected TXs. The log-log scatter plot of account degree versus probability density is shown in Fig. 3(a). The top 1% of accounts with the highest transaction frequencies account for 87.3% of all TXs. This suggests that TXs in a shard are concentrated among a small set of hotspot accounts.

Insight 1: Hotspot accounts dominate a shard's transactions, so their cross-shard interactions can serve as lightweight perception information to guide TX scheduling in other shards.

2.2 Analysis of Account Relevance Based on Graph Structure

Next, we analyze the account relevance in the account-transaction graph. To simulate consensus rounds, we replay TXs in batches of 2,000 and 10,000. We define neighbor nodes as accounts with direct TXs with a given account v_i , **K-hop accounts** as nodes that did not transact with v_i in the current round but did with its K-hop ($K > 1$) neighbors, and **unrelated accounts** as nodes that transacted with neither v_i nor its K-hop neighbors. We then measure the probability that K-hop and unrelated accounts transact with v_i in the next round in Fig. 3(b) (batch = 2000) and 3(c) (batch = 10000). The results show K-hop accounts are more likely to transact with v_i than unrelated accounts, indicating that nodes connected to a node's neighbors have a higher probability of future interactions with that node.

Insight 2: Accounts with transactional relationships exhibit similar behaviors, so we can use GNN to enhance each hotspot account's features by leveraging neighbor node information.

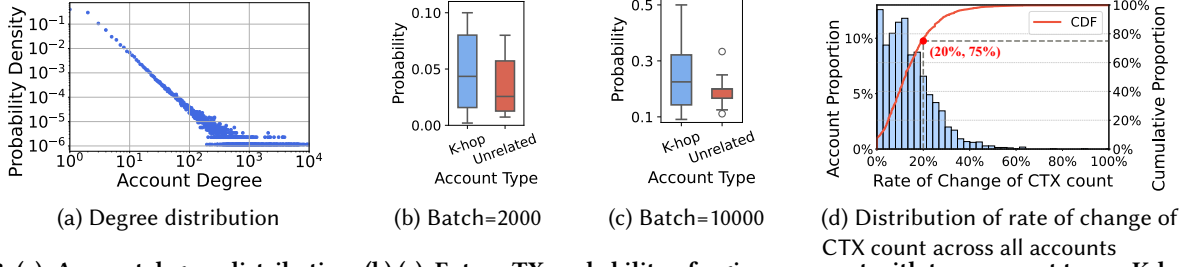


Figure 3: (a): Account degree distribution. (b) (c): Future TX probability of a given account with two account types: K-hop ($K > 1$) accounts and Unrelated accounts, for TX batch sizes of: (b) 2000 TXs, (c) 10000 TXs. (d): The distribution of the rate of change of CTX count with a single shard across all accounts.

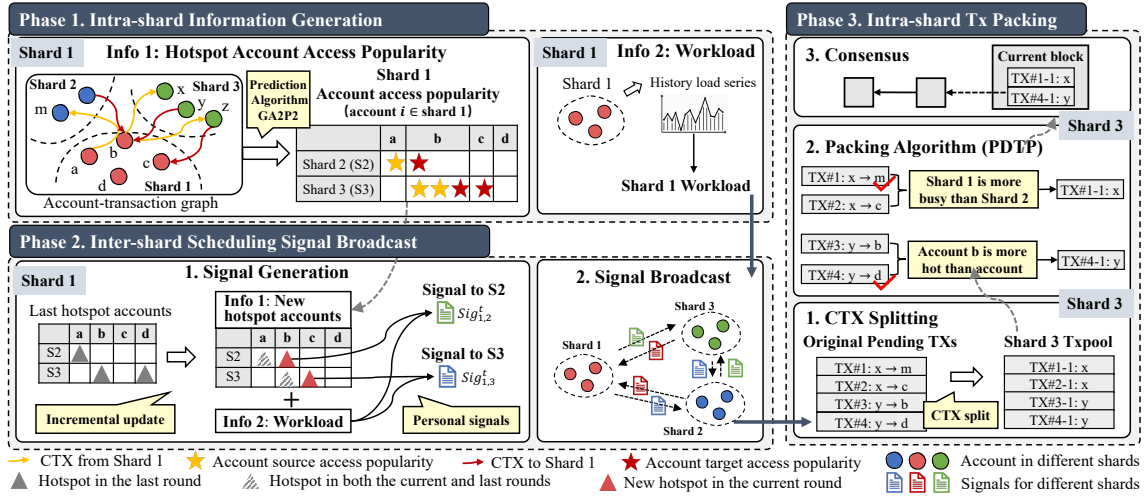


Figure 4: The workflow of Concordia.

2.3 Analysis of Cross-Shard Interaction Behaviors in Accounts

Then, we analyze the cross-shard interaction behaviors of accounts. For each account i , we track its CTX count c_i^t with a given shard in round t and compute its rate of change of CTX count $rocc_i^t$ relative to the moving average over the previous x ($x < t$) rounds:

$$\bar{c}_i^t = \frac{c_i^{t-1} + c_i^{t-2} + \dots + c_i^{t-x}}{x}, rocc_i^t = \left| \frac{c_i^t - \bar{c}_i^t}{\bar{c}_i^t} \right|. \quad (1)$$

Account i 's average $rocc_i^t$ (avg-ROCC) is the mean over all T rounds:

$$a-rocc_i = \frac{1}{T} \sum_{t=x}^T rocc_i^t, \quad (2)$$

Setting $x = 3$, we measure the avg-ROCC of all hotspot accounts and plot the distribution of the avg-ROCC of all accounts in Fig. 3(d). Results show that 75% of accounts have avg-ROCC below 20%, indicating that most accounts maintain relatively stable cross-shard interaction patterns in the short term.

Insight 3: Cross-shard interaction trends are stable in the short term. Thus, shards can use incremental or low-frequency broadcasts to share perception information, reducing communication overhead.

3 Workflow of Concordia

Concordia consists of a shard set $\mathcal{S}$ with $|\mathcal{S}|$ shards, and consensus rounds $\mathcal{T} = \{1, 2, 3, \dots\}$. TXs are either intra-shard transactions (ITXs), involving accounts within a single shard, or cross-shard transactions (CTXs), spanning two shards. Each CTX is split into a source CTX on the source shard and a target CTX on the target shard. Each shard has scheduler nodes for TX scheduling and validator nodes for handling consensus. Concordia runs in consensus rounds, each with three phases shown in Fig. 4:

Phase 1. Intra-shard Information Generation. In this phase, each shard predicts the next-round cross-shard access popularity of local hotspot accounts using a GNN-based prediction algorithm (GA2P2) in Section 4 and use it as lightweight sharing perception information. This information reflects the expected interaction trends between local accounts and external shards.

Phase 2. Inter-shard Scheduling Signal Broadcast. Based on the perception information, each shard generates personalized scheduling signals for shards expected to interact in the next round. These signals are sent to corresponding shards via an incremental personalized broadcast mechanism introduced in Section 5.

Phase 3. Intra-shard Transaction Packing. After receiving scheduling signals, each shard's scheduler uses a popularity-driven transaction packing algorithm to assemble TXs from Txpool into a block. The algorithm prioritizes TXs involving hotspot accounts

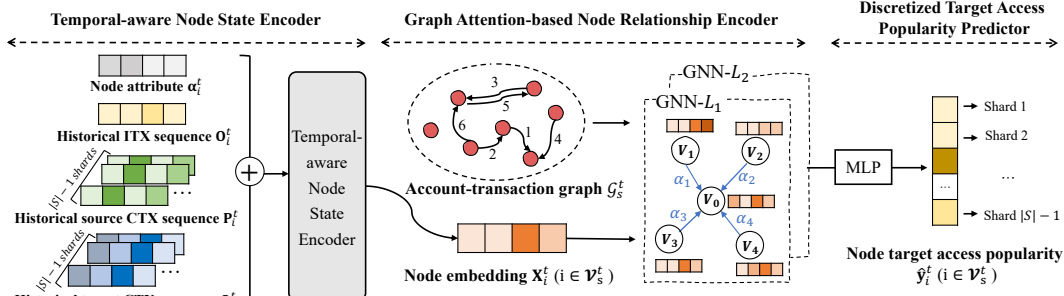


Figure 5: Model architecture of the GNN-based account target access popularity prediction algorithm (GA2P2).

based on predicted popularity, while also considering shards workload distribution. Then the scheduler sends the proposed block to all validators and run a PBFT [4] consensus to reach agreement.

4 GNN-based Account Popularity Prediction

4.1 Problem Definition

We first formalize the concepts of the popularity prediction problem.

Definition 1 (Hotspot Accounts). For the shard $s \in \mathcal{S}$ at round t , the hotspot accounts set $\mathcal{V}_s^t$ is defined as the collection of top- k accounts with the highest transaction volume in the past γ rounds.

Definition 2 (Source Access Popularity). For a hotspot account $i \in \mathcal{V}_s^t$, its source access popularity with shard $s' \in \mathcal{S} \setminus \{s\}$, denoted by $p_{i,s'}^t$, is the number of pending source CTXs sent by i to target shard s' in round $t-1$.

Definition 3 (Target Access Popularity). For a hotspot account $i \in \mathcal{V}_s^t$, its target access popularity with shard $s' \in \mathcal{S} \setminus \{s\}$, denoted by $q_{i,s'}^t$, is the number of source CTXs processed by shard s' in round t , which are initiated by shard s' and targeting account i .

Definition 4 (Account-Transaction Graph). For shard $s \in \mathcal{S}$ at round $t \in \mathcal{T}$, the Account-Transaction graph (AT-graph) is defined as a directed graph $\mathcal{G}_s^t = (\mathcal{V}_s^t, \mathcal{E}_s^t)$, where $\mathcal{V}_s^t$ is the set of hotspot accounts and $\mathcal{E}_s^t = \{(i, j) \mid i, j \in \mathcal{V}_s^t\}$ is the set of directed edges representing transactions from source account i to j .

Definition 5 (Account Target Access Popularity Prediction Problem). Given the AT-graph $\mathcal{G}_s^t$, we define $\mathbf{q}_i^t = \{q_{i,s'}^t \mid s' \in \mathcal{S} \setminus \{s\}\}$ for each hotspot account $i \in \mathcal{V}_s^t$ as its target access popularity across other shards. The goal is to predict $\mathbf{q}_i^t$ for each account $i \in \mathcal{V}_s^t$ as a multi-dimensional node regression task.

4.2 Algorithm Design

Motivated by findings in Section 2.2, we propose a GNN-based Account Access Popularity Prediction algorithm (GA2P2) for node popularity prediction, including a temporal-aware node state encoder, a graph attention-based node relationship encoder, and a discretized popularity predictor.

Temporal-aware Node State Encoder. The temporal-aware node state encoder is used for feature encoding for each hotspot node $i \in \mathcal{V}_s^t$ in shard s at round t , fusing the temporal information of each account to capture its historical transaction pattern.

First, we represent each node's attributes as $\mathbf{a}_i^t$, including its ID, the number of interacting accounts, and the creation round of the account. Next, we model each node's historical TX count

sequences with each shard, including intra-shard TX (ITX), source CTX, and target CTX. The historical ITX sequence of i is denoted as: $\mathbf{O}_i^t = \{o_{i,s'}^{t-\gamma}, \dots, o_{i,s'}^{t-2}, o_{i,s'}^{t-1}\}$, where $o_{i,s'}^t$ is the number of pending ITXs involving node i in round t and γ is the historical window size. And $\mathbf{P}_{i,s'}^t = \{p_{i,s'}^{t-\gamma}, \dots, p_{i,s'}^{t-2}, p_{i,s'}^{t-1}\}$ is the historical source CTX sequence of node i with target shard $s' \in \mathcal{S} \setminus \{s\}$ over the past γ rounds, where $p_{i,s'}^{t-1}$ is the number of source CTXs sent by i to shard s' in round $t-2$. The source CTX sequence of i with all shards in $\mathcal{S} \setminus \{s\}$ is given as $\mathbf{P}_i^t = \{\mathbf{P}_{i,1}^t, \mathbf{P}_{i,2}^t, \dots, \mathbf{P}_{i,|\mathcal{S}|-1}^t\}$. Similarly, $\mathbf{Q}_{i,s'}^t = \{q_{i,s'}^{t-\gamma}, \dots, q_{i,s'}^{t-2}, q_{i,s'}^{t-1}\}$ denotes the historical target CTX sequence of node i with source shard $s' \in \mathcal{S} \setminus \{s\}$ over the past γ rounds, and the target CTX sequence of i with all shards in $\mathcal{S} \setminus \{s\}$ is denoted as $\mathbf{Q}_i^t = \{\mathbf{Q}_{i,1}^t, \mathbf{Q}_{i,2}^t, \dots, \mathbf{Q}_{i,|\mathcal{S}|-1}^t\}$. In summary, the temporal-aware node embedding $\mathbf{X}_i^t \in \mathbb{R}^{1 \times d}$ for node i is generated by a transformer layer with self-attention [8] that integrates the above features:

$$\mathbf{X}_i^t = \text{Transformer}(\mathbf{a}_i^t \parallel \mathbf{O}_i^t \parallel \mathbf{P}_i^t \parallel \mathbf{Q}_i^t), \quad (3)$$

where $\parallel$ denotes concatenation and $\mathbf{X}^t \in \mathbb{R}^{|\mathcal{V}_s^t| \times d}$ denotes the node embedding matrix for shard s .

Graph Attention-based Node Relationship Encoder. We then further capture the spatial information for each account with a graph attention-based [35] node relationship encoder, which dynamically aggregates neighbor features. In the directed account-transaction graph $\mathcal{G}_s^t = (\mathcal{V}_s^t, \mathcal{E}_s^t)$, each hotspot account nodes i in $\mathcal{V}_s^t$ has a temporal embedding denoted as $\mathbf{X}_i^t \in \mathbb{R}^{1 \times d}$. Each edge $(i, j)_{i,j \in \mathcal{V}_s^t}$ in $\mathcal{E}_s^t$ denotes a transaction from source account i to target account j . We use $n_{i,j}^t$ to denote the number of such TXs, and $r_{i,j}^t$ to denote the round number in which the last such TX occurred between the two accounts. Let $\mathcal{N}_i$ denote the set of node i 's neighbors. Based on the above annotations, we define the node i 's representation $\mathbf{h}_i^t$ in round t by fusing spatial-temporal information on AT-graph with the following graph attention operator:

$$\mathbf{h}_i^t = \text{MLP}((1 + \epsilon) \cdot \text{BN}(\mathbf{X}_i^t) + \mathbf{h}_{\mathcal{N}(i)}^t), \quad (4)$$

$$\mathbf{h}_{\mathcal{N}(i)}^t = \sum_{j \in \mathcal{N}(i)} \alpha_{i,j}^t \cdot \mathbf{h}_j^t \mathbf{W}_n, \quad (5)$$

where $\alpha_{i,j}^t = \frac{\exp(e_{i,j}^t)}{\sum_{j' \in \mathcal{N}(i)} \exp(e_{i,j'}^t)}$ is the normalized attention weight for relation (i, j) and $\mathbf{e}_{i,j}^t = \tanh([\mathbf{X}_i^t \parallel \mathbf{X}_j^t \parallel n_{i,j}^t \parallel r_{i,j}^t] \mathbf{W}_e + \mathbf{b}_e)$ denotes the edge representation to capture the importance of a neighbor node for the center node. And $\mathbf{W}_e \in \mathbb{R}^{(2d+2) \times d_e}$, $\mathbf{b}_e \in \mathbb{R}^{1 \times d_e}$, $\mathbf{W}_n \in \mathbb{R}^{d \times d_n}$

and ϵ are learnable parameters of the graph attention layer. **BN** denotes the batch normalization function.

Finally, we use an output graph convolutional layer to get the predictions $\hat{y}_i^t \in \mathbb{R}^{1 \times (|S|-1)}$ for each central nodes as follows:

$$\hat{y}_i^t = \text{MLP}^{(2)}((1 + \epsilon^{(1)})\mathbf{h}_i^t + \sum_{j \in \mathcal{N}(i)} \alpha_{i,j}^t \cdot \mathbf{h}_j^t W_o), \quad (6)$$

where $W_o \in \mathbb{R}^{(d+d_b) \times d_o}$ is the weight matrix. $\hat{y}_i^t = \{\hat{y}_{i,1}^t, \hat{y}_{i,2}^t, \dots, \hat{y}_{i,|S|-1}^t\}$ denotes the predicted target access popularity with other shards.

Discretized Target Access Popularity Predictor. To overcome the sample sparsity challenge of continuous target prediction, we convert the continuous objective of account access count into the discrete **access popularity level** of account.

For each node $i \in \mathcal{V}_s^t$, we apply a quantile-based binning strategy to the original label vector $\mathbf{q}_i^t = \{q_{i,1}^t, q_{i,2}^t, \dots, q_{i,|S|-1}^t\}$ of target access count. Each element $q_{i,s'}^t \in \mathbf{q}_i^t$ is discretized into one of M levels and mapped to a positive integer from 1 to M as the new target label. Finally, we optimize the whole model with these discretized labels $y_i^t = \text{Quantile}(\mathbf{q}_i^t)$ by minimizing the Mean Squared Error (MSE) loss as follows:

$$\mathcal{L} = \frac{1}{|\mathcal{V}_s^t|} \sum_{i \in \mathcal{V}_s^t} (y_i^t - \hat{y}_i^t)^2. \quad (7)$$

5 Incremental Personalized Broadcast

5.1 Design overview

In Phase 2, each shard generates scheduling signals from predicted account access popularity and shares them with target shards. Since most accounts have stable short-term patterns (Section 2.3), we design an incremental personalized broadcast mechanism that only new or updated accounts are broadcast incrementally to reduce overhead. For shard $s \in \mathcal{S}$, a signal $\text{Sig}_{s,s'}^t$ is created for each $s' \in \mathcal{S} \setminus \{s\}$ in round t , containing hotspot accounts $\mathcal{A}_{s'}^t$ and shard workload W_s^t . The incremental update and broadcasting of $\mathcal{A}_{s'}^t$ are discussed in the next subsection, whereas W_s^t is derived from the Txpool records of shard s .

5.2 Incremental broadcast for popularity

The account set $\mathcal{A}_{s'}^t$ contains information on hotspot accounts in shard s that are likely to be cross-shard accessed by shard s' in the upcoming round $t \in \mathcal{T}$. First, for account $i \in \mathcal{V}_s^t$, its discretized *cross-shard access popularity* $h_{i,s'}^t$ with shard s' in round t is calculated using both source access popularity $p_{i,s'}^t$ and the predicted target access popularity level $\hat{y}_{i,s'}^t$ as follows:

$$h_{i,s'}^t = \mu \cdot \text{Quantile}(p_{i,s'}^t) + (1 - \mu) \cdot \hat{y}_{i,s'}^t, \quad (8)$$

where $\hat{y}_{i,s'}^t$ is predicted by algorithm GA2P2. Then, shard s generates the hotspot account set $\mathcal{A}_{s'}^t$ for each target shard s' . To minimize communication overhead, $\mathcal{A}_{s'}^t$ is incrementally updated and divided into two parts: a main sequence $\mathcal{M}_{s'}^t$ and an incremental sequence $\mathcal{I}_{s'}^t$. Each entry in these sequences contains a hotspot account and its corresponding *cross-shard access popularity* relative to shard s' .

The main sequence $\mathcal{M}_{s'}^t$ contains new accounts that did not appear in the previous round's account set $\mathcal{A}_{s'}^{t-1}$. For each new account i , an entry is created with an incremented index idx as the key and storing the account address $addr_i$ along with its *cross-shard access popularity* $h_{i,s'}^t$ as the value, denoted as $\langle idx, addr_i, h_{i,s'}^t \rangle$.

The incremental sequence $\mathcal{I}_{s'}^t$ contains accounts that already existed in $\mathcal{A}_{s'}^{t-1}$. We define a fluctuation threshold $(h_{i,s'}^{t-1} - \delta, h_{i,s'}^{t-1} + \delta)$ to determine popularity changes. Accounts in $\mathcal{I}_{s'}^t$ are classified into two types: 1) With significant popularity change: If an account's *cross-shard access popularity* $h_{i,s'}^t$ changes beyond the threshold, we update its value with an entry using an incremented index idx as the key and the previous index idx_i^{t-1} along with the updated popularity $h_{i,s'}^t$ as the value, denoted $\langle idx, idx_i^{t-1}, h_{i,s'}^t \rangle$. 2) With no significant popularity change: If the *cross-shard access popularity* change is within the threshold, we create an entry using the previous index idx_i^{t-1} with the popularity marked as none as the value, denoted $\langle idx, idx_i^{t-1}, none \rangle$. This allows shard s' to reuse the popularity value from the previous round's signal.

6 Heat-driven Transaction Packing

6.1 Problem Definition

We define the intra-shard transaction packing problem as follows. For shard $s \in \mathcal{S}$, the Txpool $\mathcal{N}_s^t$ in round t contains ITX set $\mathcal{O}$, source CTX set $\mathcal{P}$, and target CTXs set $\mathcal{Q}$, i.e., $\mathcal{N}_s^t = \mathcal{O} \cup \mathcal{P} \cup \mathcal{Q}$. Let block size of shard s as B_s . For each TX $k \in \mathcal{N}_s^t$, we define its attributes by transaction type below.

Conflict Degree. The conflict degree measures the likelihood that a TX k will conflict with others if packed in the current round. For account $i \in \mathcal{V}_s^t$, its *total access popularity level* is defined as:

$$H_i = \sum_{s' \in \mathcal{S} \setminus \{s\}} \text{Quantile}(p_{i,s'}^t) + \sum_{s' \in \mathcal{S} \setminus \{s\}} \hat{y}_{i,s'}^t, \forall i \in \mathcal{V}_s^t, \quad (9)$$

where $\text{Quantile}(p_{i,s'}^t)$ and $\hat{y}_{i,s'}^t$ denote the discretized source and target access popularity between account i and shard s' in round t .

Let i_s and i_t be the source and target accounts of TX k . The conflict degree of TX k is depends on the popularity of all involved accounts within the local and related external shards, defined as:

$$C_k = \begin{cases} H_{i_s} + H_{i_t}, & \text{if } k \in \mathcal{O}; \\ H_{i_s} + h_{i_t,s}^t, & \text{if } k \in \mathcal{P}; \\ h_{i_s,s}^t + H_{i_t}, & \text{if } k \in \mathcal{Q}. \end{cases} \quad (10)$$

where $h_{i_t,s}^t$ and $h_{i_s,s}^t$ are provided by external scheduling signals.

Scheduling Reward. The scheduling reward R_k represents the scheduling benefit of TX k , combining its throughput and latency gains. Let s_k be the start time of TX k , and TS the current time. The waiting time is $C_k = TS - s_k$. The reward is defined as:

$$R_k = \begin{cases} \gamma T_k + (1 - \gamma)C_k, & \text{if } k \in \mathcal{O} \cup \mathcal{Q}; \\ \gamma T_k + (1 - \gamma)\alpha_r C_k, & \text{if } k \in \mathcal{P}, \end{cases} \quad (11)$$

where T_k denotes the throughput gain ($T_k = 1$ for $k \in \mathcal{O} \cup \mathcal{Q}$, and $T_k = 0$ for $k \in \mathcal{P}$), $\alpha_r \in (0, 1)$ is a temporal discount factor, and γ is a weighting parameter. This design rewards ITXs and target CTXs that directly complete TXs.

Then, the performance utility of TX $k \in \mathcal{N}_s^t$ is defined as $f_k = \lambda_1 R_k - \lambda_2 C_k$, where $\lambda_1 + \lambda_2 = 1$. Let x_k be a binary variable, with $x_k = 1$ if TX k is packed and 0 otherwise. Furthermore, we define the inter-shard workload variance as $Var = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} (W_s^t - \bar{n}^t)^2$, where $\bar{n}^t = \sum_{s \in \mathcal{S}} W_s^t / |\mathcal{S}|$.

Problem Formulation. We formulate the shard transaction packing as an optimization problem that maximizes the utility of packed TXs while minimizing the inter-shard workload variance.

Algorithm 1: Popularity-Driven Transaction Packing Algorithm (PDTP)

Input: Txpool $\mathcal{N}_s^t$ of shard s , Block size B_s of shard s , Shard workload vector $\mathbf{W}^t = \{W_s^t | s \in \mathcal{S}\}$

Output: The result of TX packing $\mathbf{X} = \{x_k | k \in \mathcal{N}_s^t\}$

- 1 Sort $\mathcal{N}_s^t$ based on Tx's f_k in non-increasing order and select top- B_s Tx's to form the initial set $\mathcal{L}$;
- 2 **foreach** $s' \in \mathcal{S}_h$ **do**
- 3 **if** $\bar{m}_{s'} > m_{s'}$ **then**
- 4 Remove source CTXs with target shard s' from $\mathcal{L}$
- 5 **until** $\bar{m}_{s'} = m_{s'}$;
- 6 **if** $\bar{n}_{s'} < n_{s'}$ **then**
- 7 Select unpacked target CTXs with source shard s'
- 8 and add to $\mathcal{L}$ until $\bar{n}_{s'} = n_{s'}$;
- 7 **foreach** $s' \in \mathcal{S}_c$ **do**
- 8 **if** $\bar{m}_{s'} < m_{s'}$ **then**
- 9 Select unpacked source CTXs with target shard s'
- 10 and add to $\mathcal{L}$ until $\bar{m}_{s'} = m_{s'}$;
- 11 **if** $\bar{n}_{s'} > n_{s'}$ **then**
- 12 Remove target CTXs with source shard s' from $\mathcal{L}$
- 13 **until** $\bar{n}_{s'} = n_{s'}$;
- 12 $\mathbf{X} = \{x_k = 1 | k \in \mathcal{L}\}$;
- 13 **return** $\mathbf{X}$

The goal is to find a TX packing strategy $\mathbf{X} = \{x_k | k \in \mathcal{N}_s^t\}$ to maximize the following objective function:

$$\text{Max: } \sum_{k \in \mathcal{N}_s^t} f_k \cdot x_k - \text{Var} \quad (12)$$

$$\text{s.t. } x_k \in \{0, 1\}, \forall k \in \mathcal{N}_s^t. \quad (13)$$

$$\sum_{k \in \mathcal{N}_s^t} x_k \leq B_s, \forall k \in \mathcal{N}_s^t. \quad (14)$$

6.2 Scheduling Strategy

The above shard TX packing problem is NP-hard [6], so we propose a heuristic Popularity-Driven Transaction Packing algorithm (PDTP) with two phases: initial ranking, which selects TXs by maximizing performance utility, and iterative optimization, which refines the TX set to balance inter-shard workload. Algorithm 1 illustrates PDTP for a given shard s , taking Txpool $\mathcal{N}_s^t$, block size B_s , and shard workload vector $\mathbf{W}^t = \{W_s^t | s \in \mathcal{S}\}$ from scheduling signals as inputs, and outputs the TX packing result $\mathbf{X} = \{x_k | k \in \mathcal{N}_s^t\}$.

(1) Initial ranking phase. PDTP algorithm first calculates the performance utility factor f_k for each TX $k \in \mathcal{N}_s^t$, then sorted TXs in non-increasing order of utility values. The top- B_s TXs are selected to form the initial candidate set $\mathcal{L}$. (Line 1).

(2) Iterative optimization phase. Then, the algorithm adjusts the number of different types of TXs in $\mathcal{L}$ based on shard workload to achieve load balancing. Shards in $\mathcal{S} \setminus \{s\}$ are partitioned equally into a hot shard set $\mathcal{S}_h$ and a cold shard set $\mathcal{S}_c$ based on shard workload vector $\mathbf{W}^t$. PDTP algorithm determines the number of ITXs, source CTXs, and target CTXs in the current block after initial ranking, denoted as L_i , L_m , and L_n , respectively. Then, PDTP

calculates the guideline number of packed source/target CTXs associated with different shards. The guideline value of packed source CTXs with target shard $s' \in \mathcal{S} \setminus \{s\}$ is denoted as:

$$m_{s'} = L_m \cdot \frac{\frac{1}{W_{s'}^t + \epsilon}}{\sum_{\hat{s} \in \mathcal{S} \setminus \{s\}} \frac{1}{W_{\hat{s}}^t + \epsilon}}, \quad (15)$$

where $\epsilon \in \mathbb{R}^+$ is a little positive number. The guideline value of packed target CTXs with source shard $s' \in \mathcal{S} \setminus \{s\}$ is denoted as:

$$n_{s'} = L_n \cdot \frac{W_{s'}^t}{\sum_{\hat{s} \in \mathcal{S} \setminus \{s\}} W_{\hat{s}}^t}. \quad (16)$$

In the initial $\mathcal{L}$, the number of source CTXs with target shard s' is $\bar{m}_{s'}$, and the number of target CTXs with source shard s' is $\bar{n}_{s'}$.

Next, PDTP iterates over $\mathcal{S}_h$ and $\mathcal{S}_c$ to adjust TXs in $\mathcal{L}$. For each shard s' in the hot shard set $\mathcal{S}_h$, the algorithm first removes source CTXs with target shard s' from $\mathcal{L}$ if $\bar{m}_{s'}$ exceeds $m_{s'}$, and then adds target CTXs with source shard s' from the remaining Txpool if $\bar{n}_{s'}$ is below $n_{s'}$ (Lines 2-6). Similarly, for each shard s' in the cold shard set $\mathcal{S}_c$, the algorithm adds source CTXs with target shard s' if $\bar{m}_{s'}$ is below $m_{s'}$, and removes target CTXs with source shard s' if $\bar{n}_{s'}$ exceeds $n_{s'}$ (Lines 7-11).

7 Evaluation

7.1 Settings

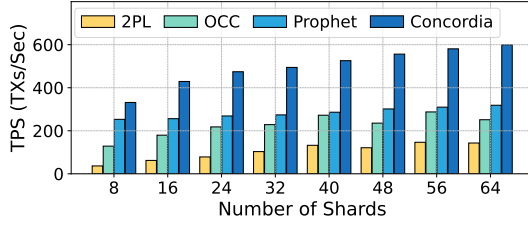
Implementation: To evaluate the performance of Concordia, we implement a Python-based prototype system like the existing studies [23, 33]. We define the retry count as the maximum number of attempts allowed for a transaction before it is aborted due to conflicts. We use one million real Ethereum [39] TXs to evaluate the overall system performance and another one million TXs to train the GA2P2 model. More details about the experiment settings are shown in Appendix B.

Baselines: We compare Concordia with three sharding systems: 1) 2PL: a non-perception sharding system Chainspace [34] with two-phase locking mechanism; 2) OCC: a non-perception sharding system Monoxide [38] with optimistic concurrency control mechanism; 3) Prophet [11]: a central-perception sharding system.

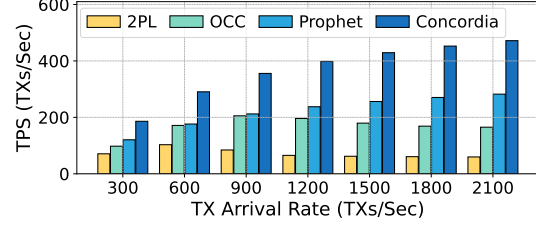
Metrics: We evaluate system performance using standard metrics such as throughput, latency, and shard load variance. In addition, we introduce two new metrics: conflict ratio of packed TXs, which measures the proportion of conflict TXs among all packed TXs, and the TX abort ratio, which measures the proportion of TXs aborted after exceeding the retry count.

7.2 Throughput

We first evaluate the average throughput as shown in Fig. 6. In Fig. 6(a), with 1500 TXs/sec and a retry count of 2, Concordia's throughput increases much faster than the other systems as the shard number grows, showing strong scalability. With 16 shards, the throughput of 2PL, OCC, Prophet, and Concordia is 62.2, 179.6, 256.2, and 429.1 TXs/sec, respectively. Concordia achieves 6.90×, 2.39×, and 1.67× higher throughput than 2PL, OCC, and Prophet. In Fig. 6(b), the throughput of 2PL and OCC peaks around 900 TXs/sec and then declines, while Concordia continues to grow faster than Prophet. At 2100 TXs/sec, the throughput of 2PL, OCC, Prophet, and

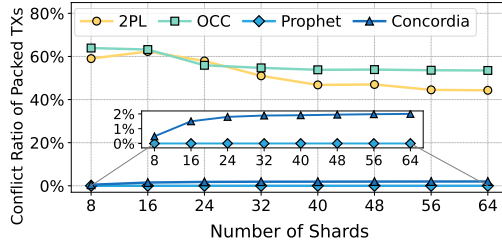


(a) TX arrival rate = 1500 TXs/Sec, retry = 2

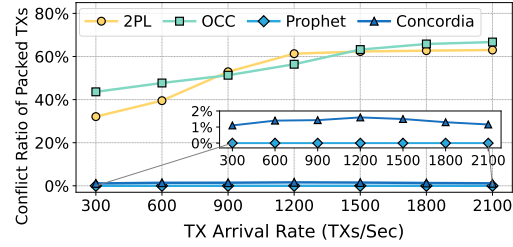


(b) Number of Shards = 16, retry = 2

Figure 6: Throughput under different numbers of shards and TX arrival rates.

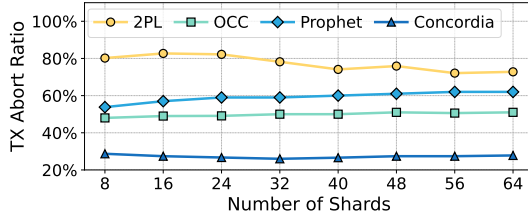


(a) TX arrival rate = 1500 TXs/Sec, retry = 2

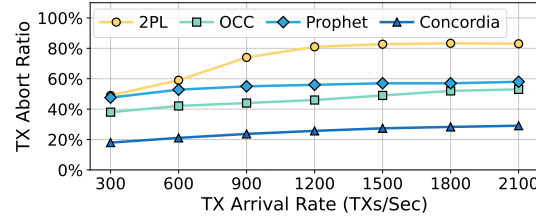


(b) Number of Shards = 16, retry = 2

Figure 7: The conflict ratio of packed TXs under different numbers of shards and TX arrival rates.



(a) TX arrival rate = 1500 TXs/Sec, retry = 2



(b) Number of Shards = 16, retry = 2

Figure 8: The TX abort ratio under different numbers of shards and TX arrival rates.

Concordia reaches 59.7, 165.2, 282.2, and 471.7 TXs/sec. Although Prophet avoids conflicts, its frequent global synchronization and centralized computation introduce significant latency.

7.3 Conflict Ratio of Packed TXs

Then, we evaluate the conflict ratio of packed TXs as shown in Fig. 7. Prophet maintains a conflict ratio of 0% across all configurations. As shown in Fig. 7(a), when varying the shard number, 2PL and OCC, show very high conflict ratios. With 16 shards, their conflict ratios exceed 60%, wasting system resources and degrading performance. In contrast, Concordia leverages decentralized cross-shard cooperative perception, significantly lowering the conflict ratio. With 16 shards, the conflict ratios of 2PL, OCC, and Concordia are 63.2%, 62.3%, and 1.5%, respectively. As shown in Fig. 7(b), when fixing the shard number, the conflict ratios of 2PL and OCC increase with higher TX arrival rates, while Concordia consistently maintains a conflict ratio below 2%. These results demonstrate that Concordia achieves efficient TX scheduling and significantly reduces conflicts.

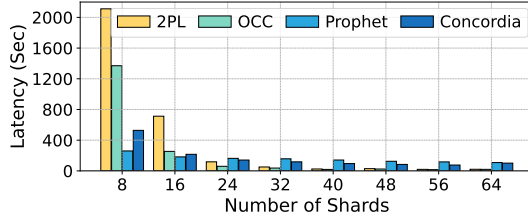
7.4 TX Abort Ratio

We evaluate the TX abort ratio as shown in Fig. 8. The TX abort ratio represents the proportion of TXs that fail to commit after

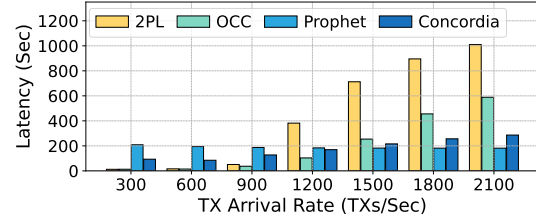
reaching the maximum retry count. In both Prophet and Concordia, an early-stop mechanism is applied to proactively halt TXs accessed hotspot accounts, preventing resource waste. These halted TXs are also included in the TX abort ratio. Fig. 8(a) presents the TX abort ratio under varying shard counts. With 16 shards, the TX abort ratio of 2PL, OCC, Prophet and Concordia are 82.7%, 49.0%, 57.1%, and 27.4%, respectively. Fig. 8(b) shows that the TX abort ratio rises for all systems as the TX arrival rate increases, due to workloads surpassing system capacity. Despite this, Concordia consistently maintains the lowest abort ratio. At an arrival rate of 1200 TXs/Sec, Concordia records a 25.7% TX abort ratio.

7.5 Latency

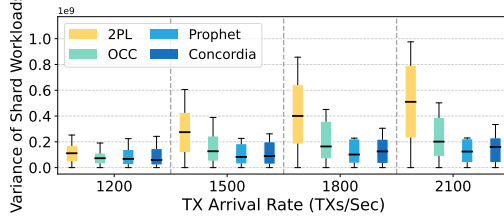
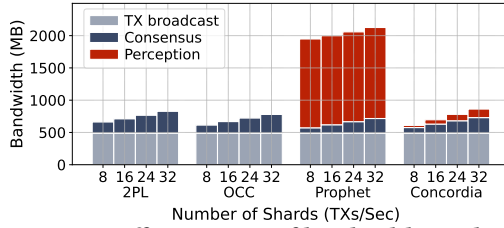
We assess transaction latency in Fig. 9. In Fig. 9(a), systems with perception capabilities achieve lower latency with fewer shards. When the shard count exceeds 24, Prophet and Concordia incur higher latency than 2PL and OCC because their scheduling strategies delay certain TXs to reduce conflicts. At 24 shards, Concordia's latency is 76.1 seconds, 35.0% lower than Prophet. In Fig. 9(b), increasing arrival rates cause latency in 2PL and OCC to rise sharply as the system becomes overloaded. In contrast, Concordia leverages cross-shard perception to schedule TXs, maintaining low latency.



(a) TX arrival rate = 1500 TXs/Sec, retry = 2



(b) Number of Shards = 16, retry = 2

Figure 9: Latency of TXs under different numbers of shards and TX arrival rates.**Figure 10: The variance of shard workload.****Figure 11: Different types of bandwidth overhead.**

At 1800 TXs/Sec, Concordia achieves a latency of 256.37 seconds, which is 43.7% lower than 2PL and 71.4% lower than OCC.

7.6 The Variance of Shard Workload

We evaluate load-balancing capability of systems using shard load variance. In Fig. 10, we continuously recorded load variance across all shards during each round under different TX arrival rates. Concordia significantly reduces shard load variance compared to 2PL and OCC. Prophet, with centralized scheduling, achieves slightly better balancing than Concordia. Importantly, as transaction rates increase, the load variance of 2PL and OCC rises sharply, while Concordia maintains a relatively stable variance. This demonstrates that Concordia's distributed scheduling effectively balances load across shards and improves overall resource utilization.

7.7 Overhead

This subsection compares the communication bandwidth overhead under varying shard numbers over 400 consensus rounds, as shown in Fig. 11. In 2PL and OCC, bandwidth is consumed by TX broadcasting and consensus. Prophet and Concordia incur additional overhead from the perception phase. With 16 shards, Concordia introduces only 9.5% extra bandwidth for cross-shard perception. Its efficient scheduling reduces TX conflicts and retries, lowering consensus overhead relative to 2PL and OCC. Concordia's total bandwidth is 2.4% lower than 2PL and only 4.0% higher than OCC, while reducing perception overhead by 95.2% compared to Prophet.

In summary, the additional bandwidth from Concordia's cross-shard perception is acceptable compared to its performance benefits.

8 Related Work

Sharding improves blockchain performance [7, 26, 30, 32, 37, 41] by enabling parallel transaction processing. Early systems like Elastico [27] support only transaction sharding, while later works such as OmniLedger [17], LB-Chain [21], Estuary [15], and LMChain [12] introduce state sharding, partitioning accounts or UTXOs to further enhance parallelism. Recent research also explores dynamic reconfiguration [3, 45], sharding consensus [20, 24, 43], and security [22]. Handling CTXs requires mechanisms ensuring consistent commitment, mainly two-phase locking (2PL) or optimistic concurrency control (OCC). 2PL, used in Omniledger [17], Chainspace [34], AHL+ [5], SSHC [25], ByShard [9], and Set *et al.* [31], locks accounts across shards and executes the CTX only after acquiring all locks. OCC, adopted by Monoxide [38], BrokerChain [13], Pyramid [10], Meepo [44], and CoChain [19].

In sharding systems, transaction scheduling determines the execution order of TXs, and unreasonable strategies can cause conflicts and performance degradation. Existing 2PL- or OCC-based sharding systems [17, 20, 23, 24, 33, 43] limit each shard's perception to its local view, which leads to high conflicts and uneven load. To address this, some works use centralized global synchronization for scheduling. For instance, Prophet [11] designates a sequence shard to order all TXs and coordinate execution across shards. While this achieves conflict-free execution, centralized perception incurs high overhead and limits scalability.

9 Conclusion

This paper presents Concordia, a efficient cross-shard transaction scheduling via cooperative perception. It integrates GNN-based information generation, personalized incremental broadcast, and popularity-driven packing. Experiments show that Concordia significantly outperforms baselines in terms of throughput, transaction conflicts and the TX abort ratio, with much lower overhead.

Acknowledgments

This work was supported in part by the National Key R&D Program of China (Grant 2023YFB2704803), the National Natural Science Foundation of China (Grant U22B2032), the Open Research Fund of The State Key Laboratory of Blockchain and Data Security, Zhejiang University and the Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing (Grant GJJ-24-025).

References

- [1] Ramesh Adhikari, Costas Busch, and Miroslav Popovic. 2025. On the Efficiency of Dynamic Transaction Scheduling in Blockchain Sharding. *arXiv preprint arXiv:2508.07472* (2025).
- [2] Faten Adel Alabdulwahhab. 2018. Web 3.0: the decentralized web blockchain networks and protocol innovation. In *2018 1st International Conference on Computer Applications & Information Security (ICCAIS)*. IEEE, 1–4.
- [3] Ting Cai, Wuhui Chen, Jianting Zhang, and Zibin Zheng. 2024. SmartChain: A dynamic and self-adaptive sharding framework for IoT blockchain. *IEEE Transactions on Services Computing* 17, 2 (2024), 674–688.
- [4] Miguel Castro and Barbara Liskov. 2002. Practical byzantine fault tolerance and proactive recovery. *ACM Transactions on Computer Systems (TOCS)* 20, 4 (2002), 398–461.
- [5] Hung Dang, Tien Tuan Anh Dinh, Dumitrel Loghin, Ee-Chien Chang, Qian Lin, and Beng Chin Ooi. 2019. Towards scaling blockchain systems via sharding. In *Proc. of the International Conference on Management of Data (SIGMOD'19)*. 123–140.
- [6] José García and Carlos Maureira. 2021. A KNN quantum cuckoo search algorithm applied to the multidimensional knapsack problem. *Applied Soft Computing* 102 (2021), 107077.
- [7] Medhasree Ghosh, Swapnil Srivastava, Apoorva Upadhyaya, Raju Halder, and Joydeep Chandra. 2025. Catalog: Exploiting joint temporal dependencies for enhanced phishing detection on ethereum. In *Proceedings of the ACM on Web Conference 2025*. 969–977.
- [8] Kai Han, An Xiao, Enhua Wu, Jianyuan Guo, Chunjing Xu, and Yunhe Wang. 2021. Transformer in transformer. *Advances in neural information processing systems* 34 (2021), 15908–15919.
- [9] Jelle Hellings and Mohammad Sadoghi. 2021. Byshard: Sharding in a byzantine environment. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2230–2243.
- [10] Zicong Hong, Song Guo, Peng Li, and Wuhui Chen. 2021. Pyramid: A layered sharding blockchain system. In *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*. IEEE, 1–10.
- [11] Zicong Hong, Song Guo, Enyuan Zhou, Jianting Zhang, Wuhui Chen, Jinwen Liang, Jie Zhang, and Albert Zomaya. 2023. Prophet: Conflict-free sharding blockchain via byzantine-tolerant deterministic ordering. In *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 1–10.
- [12] Dengcheng Hu, Jianrong Wang, Xiulong Liu, Qi Li, and Keqiu Li. 2024. LMChain: An efficient load-migratable beacon-based sharding blockchain system. *IEEE Trans. Comput.* 73, 9 (2024), 2178–2191.
- [13] HuaWei Huang, Xiaowen Peng, Jianzhou Zhan, Shenyang Zhang, Yue Lin, Zibin Zheng, and Song Guo. 2022. Brokerchain: A cross-shard blockchain protocol for account/balance-based state sharding. In *IEEE INFOCOM 2022-IEEE conference on computer communications*. IEEE, 1968–1977.
- [14] Junqin Huang, Linghe Kong, Guanjie Cheng, Qiao Xiang, Guihai Chen, Gang Huang, and Xue Liu. 2024. Advancing Web 3.0: Making Smart Contracts Smarter on Blockchain. In *Proceedings of the ACM Web Conference 2024*. 1549–1560.
- [15] Linpeng Jia, Yanxiu Liu, Keyuan Wang, and Yi Sun. 2024. Estuary: A low cross-shard blockchain sharding protocol based on state splitting. *IEEE Transactions on Parallel and Distributed Systems* 35, 3 (2024), 405–420.
- [16] David Kempe and Jon Kleinberg. 2002. Protocols and impossibility results for gossip-based communication mechanisms. In *The 43rd Annual IEEE Symposium on Foundations of Computer Science, 2002. Proceedings*. IEEE, 471–480.
- [17] Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ewa Syta, and Bryan Ford. 2018. Omniledger: A secure, scale-out, decentralized ledger via sharding. In *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 583–598.
- [18] Juanjuan Li, Rui Qin, Sangtian Guan, Jiachen Hou, and Fei-Yue Wang. 2024. Blockchain intelligence: Intelligent blockchains for web 3.0 and beyond. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 54, 11 (2024), 6633–6642.
- [19] Mingzhe Li, You Lin, Jin Zhang, and Wei Wang. 2023. Cochain: High concurrency blockchain sharding via consensus on consensus. In *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 1–10.
- [20] Meiqi Li, Xinyi Luo, Kaiping Xue, Yingjie Xue, Wentuo Sun, and Jian Li. 2024. A secure and efficient blockchain sharding scheme via hybrid consensus and dynamic management. *IEEE Transactions on Information Forensics and Security* 19 (2024), 5911–5924.
- [21] Mingzhe Li, Wei Wang, and Jin Zhang. 2023. LB-Chain: Load-balanced and low-latency blockchain sharding via account migration. *IEEE Transactions on Parallel and Distributed Systems* 34, 10 (2023), 2797–2810.
- [22] Meiqi Li, Kaiping Xue, Xinyi Luo, Wentuo Sun, David SL Wei, Qibin Sun, and Jun Lu. 2024. S³ Voting: A Blockchain Sharding Based E-voting Approach with Security and Scalability. *IEEE Transactions on Dependable and Secure Computing* (2024).
- [23] Pengze Li, Mingxuan Song, Mingzhe Xing, Zhen Xiao, Qiuyu Ding, Shengjie Guan, and Jieyi Long. 2024. Spring: Improving the throughput of sharding blockchain via deep reinforcement learning based state placement. In *Proceedings of the ACM Web Conference 2024*. 2836–2846.
- [24] Andi Liu, Yizhong Liu, Qianhong Wu, Boyu Zhao, Dongyu Li, Yuan Lu, Rongxing Lu, and Willy Susilo. 2024. CHERUBIM: A secure and highly parallel cross-shard consensus using quadruple pipelined two-phase commit for sharding blockchains. *IEEE Transactions on Information Forensics and Security* 19 (2024), 3178–3193.
- [25] Yizhong Liu, Jianwei Liu, Qianhong Wu, Hui Yu, Yiming Hei, and Ziyu Zhou. 2020. SSHC: A secure and scalable hybrid consensus protocol for sharding blockchains with a formal security framework. *IEEE Transactions on Dependable and Secure Computing* 19, 3 (2020), 2070–2088.
- [26] Taylor Lundy, Narun Raman, Scott Duke Kominers, and Kevin Leyton-Brown. 2025. NFTs as a Data-Rich Test Bed: Conspicuous Consumption and its Determinants. In *Proceedings of the ACM on Web Conference 2025*. 3451–3465.
- [27] Loi Luu, Viswesh Narayanan, Chaodong Zheng, Kunal Baweja, Seth Gilbert, and Prateek Saxena. 2016. A secure sharding protocol for open blockchains. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 17–30.
- [28] Yinbin Miao, Ziteng Liu, Hongwei Li, Kim-Kwang Raymond Choo, and Robert H Deng. 2022. Privacy-preserving Byzantine-robust federated learning via blockchain systems. *IEEE Transactions on Information Forensics and Security* 17 (2022), 2848–2861.
- [29] Silvio Micali, Michael Rabin, and Salil Vadhan. 1999. Verifiable random functions. In *40th annual symposium on foundations of computer science (cat. No. 99CB37039)*. IEEE, 120–130.
- [30] Muhammad Muzammil, Abisheka Pitumpe, Xigao Li, Amir Rahmati, and Nick Nikiforakis. 2025. The poorest man in babylon: A longitudinal study of cryptocurrency investment scams. In *Proceedings of the ACM on Web Conference 2025*. 1034–1045.
- [31] Sithu Kaung Set and Gi Seok Park. 2022. Service-aware dynamic sharding approach for scalable blockchain. *IEEE Transactions on Services Computing* 16, 4 (2022), 2954–2969.
- [32] Ruizhe Shi, Ruizhi Cheng, Yuqi Fu, Bo Han, Yue Cheng, and Songqing Chen. 2025. Centralization in the Decentralized Web: Challenges and Opportunities in IPFS Data Management. In *Proceedings of the ACM on Web Conference 2025*. 4068–4076.
- [33] Mingxuan Song, Pengze Li, Bohan Zhou, Shenglin Yin, Zhen Xiao, and Jieyi Long. 2025. AERO: Enhancing sharding blockchain via deep reinforcement learning for account migration. In *Proceedings of the ACM on Web Conference 2025*. 706–716.
- [34] Alberto Sonnino. 2018. Chainspace: A shared smart contract platform. In *Network and Distributed System Security Symposium 2018 (NDSS)*.
- [35] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al. 2017. Graph attention networks. *stat* 1050, 20 (2017), 10–48550.
- [36] Anton Wahrstätter, Jens Ernstberger, Aviv Yaish, Liyi Zhou, Kaihua Qin, Taro Tsuchiya, Sebastian Steinhorst, Davor Svetinovic, Nicolas Christin, Mikolaj Barczentewicz, et al. 2024. Blockchain censorship. In *Proceedings of the ACM Web Conference 2024*. 1632–1643.
- [37] Jiaxin Wang, Qian'ang Mao, Hongliang Sun, and Jiaqi Yan. 2025. Gamblers or Delegates: Identifying Hidden Participant Roles in Crypto Casinos. In *Proceedings of the ACM on Web Conference 2025*. 1004–1015.
- [38] Jiaping Wang and Hao Wang. 2019. Monoxide: Scale out blockchains with asynchronous consensus zones. In *16th USENIX symposium on networked systems design and implementation (NSDI)*. 95–112.
- [39] Gavin Wood. 2025. Ethereum: A secure decentralised generalised transaction ledger. <https://ethereum.github.io/yellowpaper/paper.pdf>.
- [40] Jie Xu, Yulong Ming, Zihan Wu, Cong Wang, and Xiaohua Jia. 2024. X-shard: Optimistic cross-shard transaction processing for sharding-based blockchains. *IEEE Transactions on Parallel and Distributed Systems* 35, 4 (2024), 548–559.
- [41] Xinyao Xu, Ziyu Mao, Jianzhong Su, Xingwei Lin, David Basin, Jun Sun, and Jingyi Wang. 2025. Quantitative Runtime Monitoring of Ethereum Transaction Attacks. In *Proceedings of the ACM on Web Conference 2025*. 4146–4159.
- [42] Mahdi Zamani, Mahnush Movahedi, and Mariana Raykova. 2018. Rapidchain: Scaling blockchain via full sharding. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 931–948.
- [43] Mingzhe Zhai, Yizhong Liu, Qianhong Wu, Bo Qin, Haibin Zheng, Xiaopeng Dai, Zhenyang Ding, and Willy Susilo. 2024. Accountable Secret Committee Election and Anonymous Sharding Blockchain Consensus. *IEEE Transactions on Information Forensics and Security* (2024).
- [44] Peilin Zheng, Quanqing Xu, Zibin Zheng, Zhiyuan Zhou, Ying Yan, and Hui Zhang. 2022. Meepo: Multiple execution environments per organization in sharded consortium blockchain. *IEEE Journal on Selected Areas in Communications* 40, 12 (2022), 3562–3574.
- [45] Kun Zhou, Hongbing Cheng, Zhicheng Xu, Jie Gao, and Huixin Chen. 2025. Optimizing Blockchain Shard Allocations Service: A Multi-Objective Evolutionary Perspective. *IEEE Transactions on Services Computing* (2025).

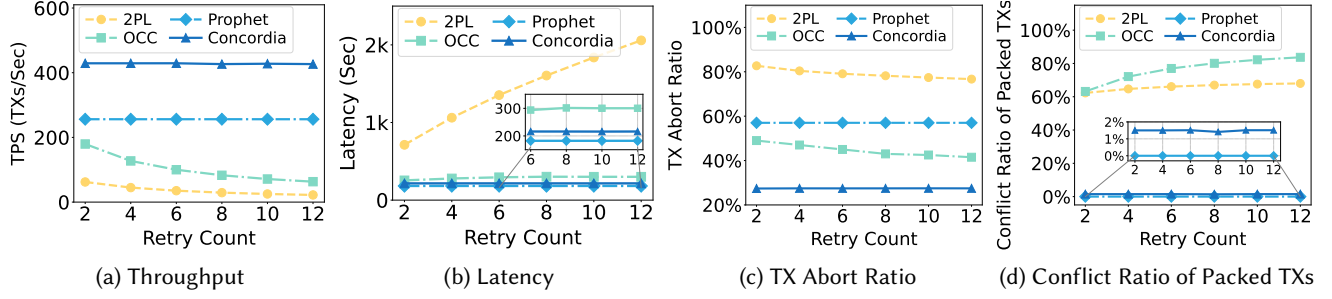


Figure 12: Throughput, latency, TX abort ratio, and conflict ratio of packed TXs under various retry counts, while the TX arrival rate is 1500 TXs/Sec, and the number of Shards is 16.

A Basic System Model

Like existing shard systems [17, 38, 42], Concordia follows a Byzantine fault-tolerant (BFT) model [4], tolerating up to one-third of nodes controlled by a mildly adaptive adversary [17]. Honest nodes strictly follow the protocol, while malicious nodes may behave arbitrarily, e.g., tampering with messages or launching denial-of-service attacks. The system assumes a partially synchronous network [4]. In Concordia, each shard has scheduler and validator nodes. The scheduler is responsible for transaction scheduling and packing each round, while validators run intra-shard consensus. All full nodes act as validators, and the scheduler is elected via a verifiable random function (VRF) [29].

B Experiment Settings

Implementation: Like the existing blockchain testbed in [10, 17, 33], we set the network bandwidth among nodes to 20 Mbps, and the synchronous period is 1 second. Nodes in our prototype use the gossip protocol [16] for communication. In the experiment, we set the block interval to 5 seconds, the maximum number of TXs per block to 500, and the transaction size to 512 bytes. After exceeding this retry count, the TX is aborted. The historical window size γ is set to 20. The parameters M , α_r , μ , and ϵ in the shard TX scheduling strategy are set to 10, 0.5, 0.5, and $1e-5$.

We implement our GA2P2 algorithm with PyTorch and Torch-Geometric. We train our GA2P2 model with 100 epochs with a batch size 16. We repeated each training five times with different random seeds for model initialization. We use the Adam optimizer for all experiments, and the initial learning rate is set to 0.001;

weight decay is set to $4e-3$. The hidden dimension of the GNN layer is set to 64.

Dataset: We utilize a real-world Ethereum [39] TX dataset comprising 2 million historical TXs from Jan. 15, 2024 (from block height: 19,010,000). We run the shard prototype system and execute the first million TXs in the Ethereum TX dataset. We recorded the account-transaction graph data during each round to build a dataset for training the GA2P2 algorithm. The node labels in each round’s account-transaction graph utilize the target access popularity in the subsequent round. The GA2P2 algorithm training dataset comprises a collection of account-transaction graphs across all rounds in all shards. The remaining one million TXs are used to evaluate the overall performance of the sharding systems.

C Experiment on Varying TX Retry Count

In this part, we evaluate the system performance under different TX retry counts as shown in Fig. 12. In Concordia, its TX scheduling algorithm uses cross-shard perception to adjust the suitable processing order of TXs to minimize conflicts, resulting in a very low conflict ratio of packed TXs (around 2%). As a result, most TXs are successfully committed on the first try. In Prophet, the conflict ratio of packed TXs is 0, so all TXs can be committed in the first attempt. As the retry count increases, Concordia and Prophet maintain stable performance across all metrics, including throughput, latency, TX abort ratio, and conflict ratio of packed TXs. However, in 2PL and OCC, while the TX abort ratio decreases with more retries, other metrics like throughput, latency, and conflict ratio of packed TXs worsen.